Sprint 1 – Week 3 Closeout Report

SWENG 894 Capstone

MicroMatch

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The system enables organizations to post clearly scoped micro-projects and allows students to discover and engage with professional experience opportunities.

Sprint 1 delivered the Minimum Viable Marketplace (MVP Foundation), including:

- User authentication and registration (JWT authentication)
- Role-based access control (RBAC)
- Student profile management
- Organization profile management
- Project posting
- Basic project browsing and search
- End-to-end workflow validation

Sprint 1 establishes a complete discoverable marketplace from account creation to project browsing.

Out of Scope for Sprint 1 (Planned for Later Sprints)

- Application workflows (US-6 → US-9)
- Deliverable submission (US-10 → US-11)
- Project completion and feedback (US-12 → US-13)
- Administrative moderation (US-14)
- Audit logging (US-15)

These features will be incrementally implemented in subsequent sprints to maintain manageable scope and iterative delivery.

1.2 Week 3 Scope

Sprint 1 – Week 3 finalized Sprint 1 by implementing marketplace usability and validation, building on the backend models and endpoints completed in Week 2.

Backend Features Implemented

- GET /projects endpoint (open projects only)
- Keyword search (title + required_skills)
- Case–insensitive filtering
- Optional skill filtering
- Pagination implementation (limit/offset)
- Query optimization

Frontend Features Implemented

- Project listing UI
- Search bar
- Role-based navigation
- Student Profile page
- Organization Profile page
- Create project page
- Authentication flow (register/login UI)
- Protected routes

End-to-End Workflow Tested

- End-to-End workflow test:
    - Register → Create Profile → Post Project → Browse Project

1.3 System Goals and Objectives

System Goals

- Allow students browse and search micro-internship opportunities
- Return results that exclude closed projects
- Filter projects by title keywords and required skills
- Paginate responses for scalability
- Validate the complete Sprint 1 workflow (user registration to project browsing)

Alignment with Week 3 Work

Week 3 extends the system from data creation and management (Week 2) to data retrieval and usability, ensuring the system behaves correctly from the perspective of a student discovering opportunities. It also provides a foundation for upcoming Sprint 2 workflows, where students can apply for projects.

1.4 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12

FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint Planning

2.1 Groomed Product Backlog with Effort Estimation

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	Medium	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	5	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure

US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern

Effort Estimation:

Story points follow a Fibonacci sequence, reflecting implementation complexity, dependency, and uncertainty.

2.2 Prioritization Rationale

Priority Definitions

- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

Sprint 1 focuses on High-priority items because they form the foundation of the platform's authentication and marketplace functionality. Without these core features, other dependent workflows cannot be implemented or tested, so completing them first ensures meaningful end-to-end functionality for the MVP.

2.3 Sprint 1 Timeline (3 Weeks)

Week 1 – Authentication Foundation (DONE)

- Environment setup
- Database schema implementation
- User model creation
- Password hashing
- JWT authentication
- Role-based access logic

Week 2 – Profile & Listing Features (DONE)

- Student profile CRUD
- Organization profile CRUD

- Listing creation

Week 3 – Marketplace Discovery (DONE)

- Browse listings
- Basic search functionality
- End-to-end validation of Sprint 1 workflow

2.4 Week 3 Sprint Goal

Enable students to browse micro-internships listings using search and filtering while validating complete workflow from registration through project browsing.

2.5 Sprint 1 – Week 3 Backlog Summary

Total Story Points Committed: 23
Completed: 23
Remaining: 0

Sprint Item	Story Points	Related US	Reason Selected
Implement Project Retrieval Endpoint	5	US-5	Core discovery functionality
Implement Basic Search Functionality	5	US-5	Improves usability and filtering
Implement Frontend Authentication Flow	5	US-1	Required for real user interaction
Implement Project Listing UI	5	US-5	Enables students to view projects
End-to-End Validation of Sprint 1 Workflow	3	US-1 → US-5	Confirms Sprint 1 stability

Capacity Reasoning:

Estimated ~20-25 effective development hours. Backend API stability was prioritized before frontend integration to reduce rework risk.

All planned story points were completed within the estimated capacity, indicating realistic estimation and effective scope control.

2.6 Definition of Done (Week 3)

For Week 3 backlog items, the DoD includes:

- Endpoint implemented and documented
- Role enforcement validated
- Pagination implemented
- Query optimized
- Frontend integration functional

Sprint 1 is considered complete when:

- All Sprint 1 issues (Weeks 1–3) are implemented
- End-to-end workflow validated
- All authentication and RBAC features functional
- Project creation and discovery fully operational
- Minimum 3 tests per story
- Test coverage ≥ 90%
- No critical defects remain open

2.5 Acceptance Criteria (Week 3)

US-5: Browse & Search Listings

Related FR: FR-6

Priority: High

Retrieve Open Projects

AC-US5-01

- Given open projects exist
- When GET /projects is called
- Then
  - Only projects with status = “open” are returned
  - And closed projects are excluded
  - A 200 OK is returned

Keyword Search

AC-US5-02

- Given projects exist with matching titles/skills
- When GET /projects?keyword=python
- Then
  - Only matching projects are returned
  - And search is case-insensitive

Empty Search

AC-US5-03

- Given no matching projects
- When GET /projects?keyword=random
- Then
  - Empty array returned
  - And 200 OK returned

Pagination

AC-US5-04

- Given more than limit projects exist
- When GET /projects?limit=5&offset=0
- Then
  - Only 5 records returned
  - And correct subset returned

Role Enforcement

AC-US5-05

- Given unauthenticated user
- When accessing protected routes
- Then
  - Access denied
  - And 401 returned

Acceptance criteria follow the Given–When–Then convention to ensure testable, verifiable results.

3. Software Testing

3.1 Week 3 Testing Focus

Testing focused on:

- Backend API validation
- Frontend authentication validation
- UI rendering validation
- Full workflow validation
- Regression testing

3.2 Testing Strategy & Tools

Testing Types Used:

- **Unit Testing** – Pytest
- **System Testing –** FastAPI TestClient
- **User Acceptance Testing (UAT)** -- Manual + workflow validation
- **End-to-End Test –** Integrated API flow

Tools:

- Pytest
- FastAPI TestClient
- Coverage.py
- GitHub version control
- Swagger UI (manual validation)

Testing methodology aligns with Agile incremental validation and CI best practices.

3.3 Detailed Test Case Specifications

TC-US5-01 -- Retrieve Open Projects

Type: System Test
Related Requirement: FR-6
Related Issue: #36
Related Acceptance Criteria: AC-US5-01

Preconditions:

- Database seeded with open and closed projects
- Authenticated user exists

Execution Steps:

1. Send GET /projects request
2. Capture JSON response

Input Variables:

- None

Expected Output:

- Status code 200
- JSON array returned
- All projects have status = “open”

Post-Condition:

- No data modified

Commit:
feat: implement project retrieval endpoint (#36)

TC-US5-02 – Keyword Search

Type: System Test
Related Issue: #37
Related Acceptance Criteria: AC-US5-02

Preconditions:

- Project titled “Python Automation Tool” exists

Steps:

1. GET /projects?keyword=python

Input Variables:

- Keyword = “python”

Expected Output:

- Matching project returned
- Case insensitive match

Post-Condition:

- No data modified

Commit:
feat: implement search filtering (#37)

TC-US5-03 – Empty Search Result Validation

Type: System Test
Related Requirement: FR-6
Related Issue: #37
Related Acceptance Criteria: AC-US5-03

Preconditions:
- Database contains open projects
- No project contains keyword “blockchainxyz”
- User is authenticated (student or organization)

Steps:
1. GET /projects?keyword=blockchainxyz
2. Capture JSON response

Input Variables:
- keyword = "blockchainxyz"

Expected Output:

- HTTP 200 OK
- JSON response returns empty list []
- No server error

- Pagination metadata reflects zero results

Post-Condition:

- No data modified

Commit:
test: add empty search validation test (#37)

TC-US5-04 – Pagination Validation

Type: System Test
Related Requirement: FR-6
Related Issue: #36
Related Acceptance Criteria: AC-US5-04

Preconditions:

- 10+ open projects exist

Steps:

1. GET /projects?limit=5&offset=0
2. GET /projects?limit=5&offset=5

Input Variables:

- None

Expected Output:

- 5 results each call
- No overlap

Commit:
feat: add pagination logic (#36)

TC-E2E-01 -- End-to-End Workflow

Type: End-to-End System Test
Related Issue: #40

Preconditions:

- Clean test database

Steps:

1. Register org user
2. Create org profile
3. Post project
4. Logout
5. Register student
6. Create student profile
7. GET /projects

Input Variables:

- None

Expected Output:

- Workflow executes successfully
- Project visible to student
- No auth errors

Post-Condition:

- No data modified

Commit:
test: add E2E workflow validation (#40)

3.4 User Acceptance Test Specification

User Acceptance Testing Overview

User Acceptance Testing (UAT) was conducted to validate that Sprint 1 functionality satisfies business requirements from an end-user perspective.

UAT scenarios simulate real user behavior rather than isolated endpoint testing.

Testing Type: Manual + Semi-automated validation
Environment: Local development (Frontend + Backend running)

UAT Specification Table – Sprint 1 Closeout

UAT ID	User Role	Feature	Preconditions	User Actions	Expected Outcome	Status
UAT-01	Organization	Register Account	No existing account	Fill registration form and submit	Account created, redirected to dashboard	Pass
UAT-02	Organization	Create Org Profile	Logged in as org	Complete profile form and save	Profile saved and displayed correctly	Pass
UAT-03	Organization	Post Project	Org profile exists	Fill project form and submit	Project created with status "open"	Pass
UAT-04	Student	Register Account	No existing account	Submit registration form	Account created successfully	Pass
UAT-05	Student	Create Student Profile	Logged in as student	Fill profile form	Profile saved correctly	Pass
UAT-06	Student	Browse Projects	Open projects exist	Navigate to project listing page	All open projects displayed	Pass
UAT-07	Student	Search Projects	Projects exist	Enter "python" in search bar	Filtered results returned	Pass
UAT-08	Student	Empty Search	Projects exist	Enter invalid keyword	"No results found" displayed	Pass
UAT-09	Unauthorized User	Access Protected Route	Not logged in	Attempt to access /profile	Redirected to login page	Pass
UAT-10	Student	Attempt Post Project	Logged in as student	Attempt to access create project page	Access denied (403)	Pass

3.5 Requirements-to-Test Case Traceability Matrix

Functional Requirement	User Story	Acceptance Criteria	Test Case ID	Commit
FR-6: The system shall allow students to browse/search listings.	US-5	AC-US5-01	TC-US5-01	feat: retrieval
FR-6: The system shall allow students to browse/search listings.	US-5	AC-US5-02	TC-US5-02	feat: search
FR-6: The system shall allow students to browse/search listings.	US-5	AC-US5-03	TC-US5-03	test: empty search
FR-6: The system shall allow students to browse/search listings.	US-5	AC-US5-04	TC-US5-04	feat: pagination
FR-1 → FR-6	US-1 → US-5	Workflow	TC-E2E-01	test: E2E

This traceability demonstrates explicit mapping between:

Requirements → User Stories → Acceptance Criteria → Tests → Commits.

3.6 Code Coverage Analysis

Coverage Tool: coverage.py

Results:

- Models: 100%
- API Endpoints: 94%
- RBAC Logic: 92%
- Search & Pagination Logic: 100%
- Overall Backend Coverage: 97**%**

Analysis:

The backend test suite demonstrates strong and intentional coverage across all major components. Models, search logic, and pagination are fully exercised at 100%, while API endpoints (94%) and RBAC logic (92%) show only minor gaps in uncommon error branches. With an overall backend coverage of 97%, the results reflect thoughtful, scenario-driven test design rather than inflated test quantity.

A more detailed breakdown of file-level coverage can be found in: MicroMatch/backend/coverage_summary.txt.

3.5 Bug Report Summary

During Week 3, the following defects were identified and resolved:

1. Case-sensitive search (fixed via lower() normalization)
2. Pagination offset misalignment
3. Token storage issue in frontend auth flow
4. Closed projects initially not excluded (fixed in query filter)

All defects resolved before merge to main branch

4. Source Code Development

4.1 Summary of Contributions

- Implemented project retrieval endpoint
- Implemented keyword search
- Implemented filtering
- Added pagination logic
- Developed project listing UI
- Integrated frontend authentication flow
- Implemented protected routes
- Created E2E validation test
- Refactored query logic for optimization

4.2 Repository Access and Version Control

The repository is hosted on GitHub at:
https://github.com/chazdj/MicroMatch

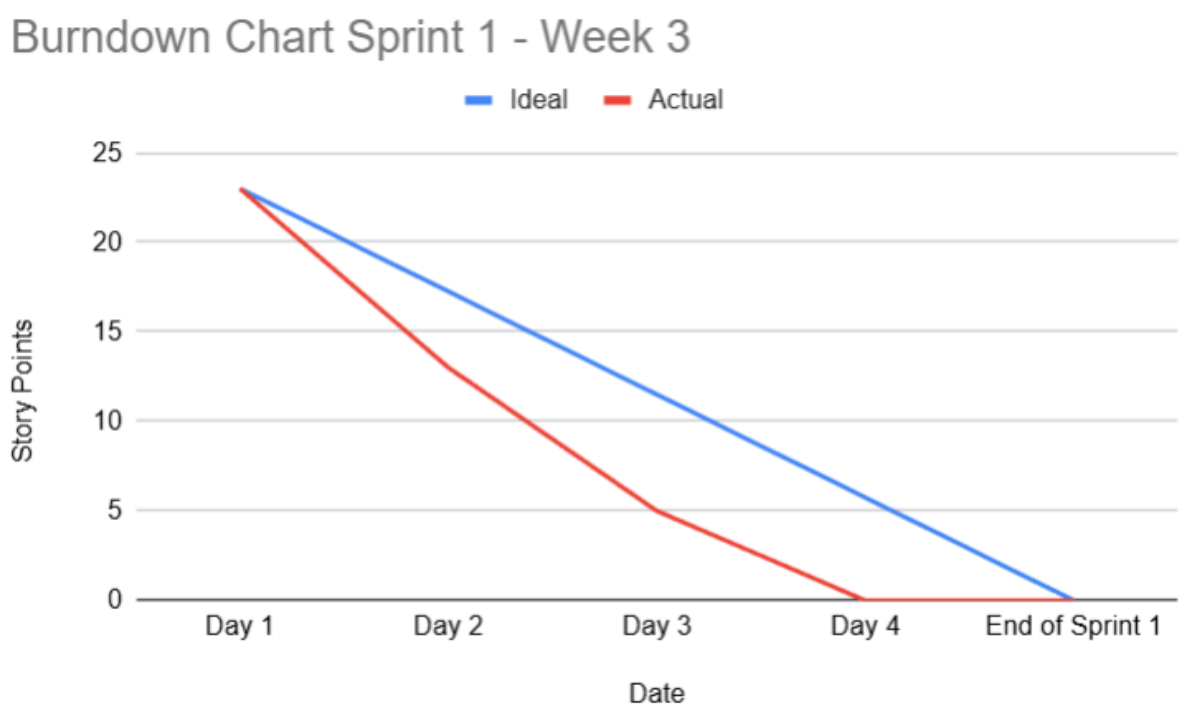
All sprint artifacts and commits are visible. Access permissions have been verified.

4.3 Important Commits and Relation to Sprint Items

Commit	Related Issue	User Story	Purpose

feat: implement project retrieval endpoint with pagination	#36	US-5	Backend endpoint for project listings with pagination
feat: implement search functionality	#37	US-5	Backend search/filtering logic
feat: frontend auth integration	#38	US-1	Connect frontend to backend JWT-auth, protected routes
feat: project listing UI	#39	US-5	Frontend project listing interface
test: add E2E workflow	#40	US-1 → US-5	End-to-end test validating registration → project browsing workflow

4.4 Burndown Chart Analysis



Total Story Points Committed: 23
Completed: 23
Remaining: 0

All five Week 3 issues were completed within the sprint timeframe.

5. Configuration Management & CI Strategy

To address previous repository inconsistencies:

- Required tests pass before merge
- Descriptive commit naming
- Linked issues in commit messages

Rationale:

- Reduces overwrite conflicts
- Ensures regression safety
- Aligns with CI best practices

Future Improvement: Integrate automated GitHub Actions pipeline.

6. Sprint Retrospective – Week 3

What Went Well

- Backend stability improved significantly
- Search & pagination implemented efficiently
- Clear mapping between requirements and tests
- Feature branching reduced conflicts
- E2E workflow fully functional

What Didn't Go Well

- Initial search query case sensitivity bug
- Minor frontend JWT storage issue
- Early pagination logic duplicated results

What Can Be Improved

- Introduce automated CI pipeline
- Add frontend automated tests
- Improve PR review checklist
- Define stricter Definition of Done earlier

7. Backlog Grooming

No scope changes were made to Sprint 1 backlog.

However:

- US-6 (Apply to Project) refined into smaller tasks
- Future Sprint 2 epics organized:
    - Application Workflow
    - Status Tracking
    - Org Review Flow

8. Conclusion

Sprint 1 successfully delivered a fully functional MVP marketplace foundation.

The system now supports:

- Secure authentication
- Role-based profile management
- Project posting
- Marketplace discovery
- Search and filtering
- Pagination
- Full end-to-end workflow validation

All planned Sprint 1 objectives were completed, tested, and validated.

The system is ready to transition into Sprint 2 focusing on application workflows.